Chapter 1: JavaScript Variables

1.1 What is a Variable?

A program needs to remember things. A game remembers your score. A shopping app remembers what is in your cart.

In JavaScript, we remember things using a **variable**. A variable is like a box with a name written on it. The name is the label, and the thing inside is the value.

To create a variable, we write a keyword, a name, an equals sign(=), and a value:

```javascript
let age = 20;
```

Here, let is the keyword, age is the name, and 20 is the value.

JavaScript has three keywords to create variables: **var, let, and const**.

1.2 What Is Scope?

Scope means: **where in the program can I use this variable?**

For now, remember two simple ideas:

- **Function scope:** the variable can be used anywhere inside the function.
- **Block scope:** the variable can be used only inside the curly braces { } where it was created.

A block is any code written inside { }, like an if statement etc.

1.3 var: The Old Way

Before 2015, var was the only way to create a variable. It still works, but it has a problem: it does not respect blocks.

Example 1: var escapes the block

```javascript
if (true) {
  var city = "Patna";
}
console.log(city);   // Output: Patna
```

We made city inside the { }, but we could still use it outside. This can cause confusion in bigger programs.

Example 2: var allows the same name twice

```javascript
var user = "Amit";
var user = "Rohan";
console.log(user);   // Output: Rohan
```

JavaScript gives no warning, so you can lose a value by mistake.

1.4 Why Were let and const Added?

In 2015, JavaScript added let and const to fix the problems of var:

1. var escapes blocks, but let and const stay inside their block.
2. var allows the same name twice, but let and const do not.
3. var has no way to say "this value must never change", but const does.

1.5 let: A Value That Can Change

Use let when the value will change later.

Example 1: Changing the value

```javascript
let score = 10;
console.log(score);   // Output: 10

score = 25;
console.log(score);   // Output: 25
```

Example 2: let stays inside its block

```javascript
if (true) {
  let message = "Hello";
  console.log(message);   // Output: Hello
}
console.log(message);     // Error: message is not defined
```

Outside the block, message does not exist.

1.6 const: A Value That Does Not Change

const means constant. Use it when the value should never change. You must give it a value when you create it.

Example 1: A constant value

```javascript
const country = "India";
console.log(country);   // Output: India
```

Example 2: Changing a constant gives an error

```javascript
const pi = 3.14;
pi = 3.15;   // Error: Assignment to constant variable
```

const with Arrays

If you store an array in const, you cannot replace the array, but you can add items to it.

```javascript
const fruits = ["apple", "banana"];
fruits.push("mango");
console.log(fruits);   // Output: ["apple", "banana", "mango"]
```

1.7 Comparison Table

Feature	`var`	`let`	`const`
Scope	Function	Block	Block
Can change value	Yes	Yes	No
Can declare same name twice	Yes	No	No
Needs a value at creation	No	No	Yes
Use today?	Avoid	Yes	Yes

Rules for Naming Variables

1. Use letters, digits, _ and $.
2. Do not start with a digit (1name is wrong).

3. Do not use spaces (user name is wrong, userName is right).
4. Names are case-sensitive (age and Age are different).
5. Do not use reserved words like let, if, class.
6. Use camelCase: firstName, totalPrice.
7. Use meaningful names: price is better than p.

```javascript
let name = "Rahul";      // text (string)
let age = 20;            // number
let isStudent = true;    // true or false (boolean)
let marks;               // no value yet (undefined)
```

Five Practice Examples

Practice 1: Student Profile

Question:
Write a program to store a student's name and age in variables and display them on the console.

Practice 2: Shopping Total

Question:
Write a program to calculate the total cost of 3 items when each item costs ₹250 and display the total amount.

Practice 3: Bank Balance

Question:
Write a program to store a person's bank balance and withdrawal amount, then calculate and display the remaining balance.

Practice 4: Game Score

Question:
Write a program to calculate a player's total score after scoring 50 points and then 30 points. Display the final score.

Practice 5: Shopping List

Question:
Write a program to create a shopping list containing "milk" and "bread", then add "eggs" to the list and display the updated list.

Answers:

Practice 1: Student Profile

```javascript
const studentName = "Priya";
let studentAge = 19;

console.log(studentName, studentAge);
// Output: Priya 19
```

Practice 2: Shopping Total

Three items cost ₹250 each.

```javascript
const itemPrice = 250;
let quantity = 3;
let total = itemPrice * quantity;

console.log("Total: ₹" + total);
// Output: Total: ₹750
```

Practice 3: Bank Balance

A person has ₹5000 and withdraws ₹1200.

```javascript
let balance = 5000;
const withdraw = 1200;

balance = balance - withdraw;

console.log("Balance: ₹" + balance);
// Output: Balance: ₹3800
```

Practice 4: Game Score

A player starts with 0 points and scores twice.

```javascript
let points = 0;

points = points + 50;
points = points + 30;

console.log("Points:", points);
// Output: Points: 80
```

Practice 5: Shopping List

```javascript
const shoppingList = ["milk", "bread"];

shoppingList.push("eggs");

console.log(shoppingList);
// Output: ["milk", "bread", "eggs"]
```

Chapter 2: Hoisting

2.1 What Is Hoisting?

Normally, you must create a variable before you use it. But JavaScript has a special behavior called **hoisting**.

Before running your code, JavaScript reads through it and notices all the variables. It acts as if all declarations were moved to the top. This is called hoisting.

Important: only the **name** is moved up, not the **value**.

```javascript
console.log(x);   // Output: undefined
var x = 5;
```

JavaScript sees the code like this:

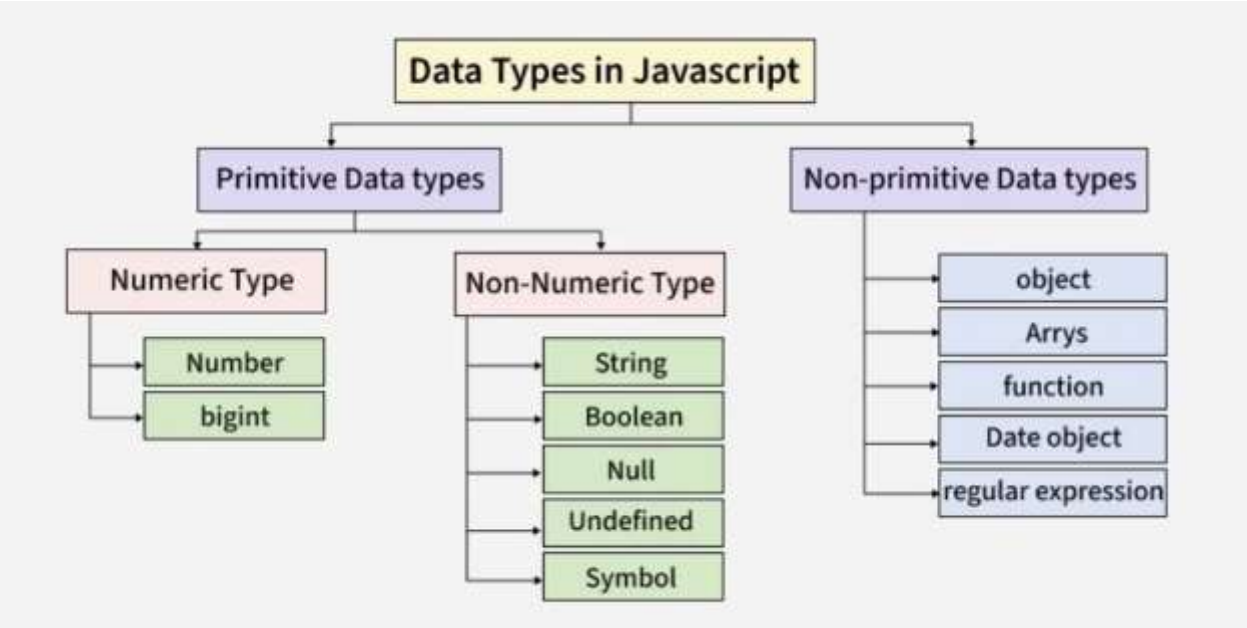
```javascript
var x;              // name moved to the top
console.log(x);     // no value yet, so undefined
x = 5;              // value is given here
```

Chapter 3: JavaScript Data Types

3.1 What Is a Data Type?

JavaScript data types define what kind of values a variable can hold and how those values behave in a program. They determine how data is stored in memory and how operations like comparison, calculation, and conversi

JavaScript Data Type Categories



there are **7** primitive data types: **string**, **number**, **boolean**, **undefined**, **null**, **symbol**, **bigint**.

Primitive data types are the most basic, built-in building blocks in programming that store a single, simple value directly in memory, whereas **non-primitive data types** (also called reference or derived types) are more complex structures built from primitive types to hold collections of values or objects.

Primitive Example:

```
// Number
let length = 16;
let weight = 7.5;
```

```
// BigInt
let x = 1234567890123456789012345n;
let y = BigInt(1234567890123456789012345);

// Strings
let color = "Yellow";
let lastName = "Johnson";

// Boolean
let x = true;
let y = false;

// Undefined
let x;
let y;

// Null
let x = null;
let y = null;

// Symbol
const x = Symbol();
const y = Symbol();

// Object
const person = {firstName:"John", lastName:"Doe"};

// Array Object
const cars = ["Saab", "Volvo", "BMW"];

// Date Object
const date = new Date("2022-03-25");
```

Non-Primitive Example:

Object

JavaScript objects are key-value pairs used to store data, created with { } or the new keyword. They are fundamental as nearly everything in JavaScript is an object.

```
let student = {
   name: "John Doe",
   location: "Noida"
```

}

console.log(student.type);

Arrays

An Array is a special kind of object used to store an ordered collection of values, which can be of any data type.

let arr1 = [1, 2, 3, 4, 5];

console.log(arr1);

let arr2 = [1, "two", 3, 4, 5]];

console.log(arr2);

Function

A function in JavaScript is a block of reusable code designed to perform a specific task when called.

// Defining a function to greet a user

```
function greet(name){
return "Hello, " + name + "!";
}
// Calling the function
console.log(greet("Ajay"));

function sum( ){
let num1 = 20;
let num2 = 30;
let sum = num1 + num2;
}
// Calling the function
console.log(“Sum of Two Number is : ” +  sum +);
```

Chapter 4: JavaScript Operators

JavaScript operators are symbols or keywords used to perform operations on values and variables. They are the building blocks of JavaScript expressions and can manipulate data in various ways.

There are some Operators category below:

Assignment Operator (=) : This operator is use for assign a value in variable.

Arithmetic Operators (+, -, *, / etc.)

Comparison Operators (> < == != etc.)

Logical Operators (&& || !)

Conditional Operators (ternary operator)

Arithmetic Operators

Arithmetic operators perform arithmetic operations on numbers.

Operator	Description
+	Addition
-	Subtraction
*	Multiplication
**	Exponentiation (ES2016)
/	Division
%	Modulus (Remainder)
++	Increment
--	Decrement

Note:

" + " is also use for joining two variables or text and variables (which is called Concatenation)

Ex: let name = "Mark";
console.log("Hi, " + name);

Questions :

(A) Rahul scored 80 marks in Maths and 75 marks in Science. Find his total marks.

(B) A bank account has ₹5000. The owner spends ₹1200. How much money is left?
(C) One pencil costs ₹5. Find the cost of 12 pencils.
(D) A student scored 80 in Test 1 and 90 in Test 2. Find the average.
(E) Check whether the number 7 is even or odd using the remainder operator.
(F) A website counter starts at 0. Three people visit the website one after another. What is the visitor count at the end?
(G) A player starts a game with 3 lives. The player loses one life. How many lives are left?

****

(H) You order three items priced ₹250, ₹399, and ₹120. The delivery charge is ₹40. Find the final bill.
(I) A phone's original price is ₹18000. In a sale it costs ₹15500, and you also have a coupon worth ₹500. Find the final price you pay and the total money you save.
(J) A room is 15 feet long and 8 feet wide. Tiles cost ₹40 per square foot. Find the total cost of tiling the floor. *(Hint: area = length × width.)*
(K) Four friends have dinner. The food bill is ₹1800 and they also leave a ₹100 tip. If they split everything equally, how much does each person pay?
(L) A post has 98 likes. Three people like it one after another. Print the number of likes after each person.
(M) A shop has 5 bottles in stock. Three customers buy one bottle each, one after another. Print the stock after each sale.

(A)

```javascript
let maths = 80;
let science = 75;

let total = maths + science;
console.log("Total marks:", total);
// Output: Total marks: 155
```

(B)

```javascript
let balance = 5000;
let spent = 1200;

let remaining = balance - spent;
console.log("Remaining: ₹" + remaining);
// Output: Remaining: ₹3800
```

(C)

```javascript
const pencilPrice = 5;
let quantity = 12;

let cost = pencilPrice * quantity;
console.log("Total: ₹" + cost);
// Output: Total: ₹60
```

(D)

```javascript
let test1 = 80;
let test2 = 90;

let average = (test1 + test2) / 2;
console.log("Average:", average);
// Output: Average: 85
```

(E)

```javascript
let number = 7;

let remainder = number % 2;
console.log("Remainder:", remainder);
// Output: Remainder: 1
// Remainder is 1, so 7 is odd.
```

(F)

```javascript
let visitors = 0;

visitors++;
visitors++;
visitors++;

console.log("Visitors:", visitors);
// Output: Visitors: 3
```

(G)

```javascript
let lives = 3;

lives--;
console.log("Lives left:", lives);
// Output: Lives left: 2
```

(H)

```javascript
const item1 = 250;
const item2 = 399;
const item3 = 120;
const delivery = 40;

let finalBill = item1 + item2 + item3 + delivery;
console.log("Final bill: ₹" + finalBill);
// Output: Final bill: ₹809
```

(I)

```javascript
const originalPrice = 18000;
const salePrice = 15500;
const coupon = 500;

let finalPrice = salePrice - coupon;
let totalSaved = originalPrice - finalPrice;

console.log("Final price: ₹" + finalPrice);
// Output: Final price: ₹15000
console.log("Total saved: ₹" + totalSaved);
// Output: Total saved: ₹3000
```

(J)

```javascript
const length = 15;
const width = 8;
const tilePrice = 40;

let area = length * width;
let tilingCost = area * tilePrice;

console.log("Area:", area);
// Output: Area: 120
console.log("Tiling cost: ₹" + tilingCost);
// Output: Tiling cost: ₹4800
```

(K)

javascript

```javascript
const foodBill = 1800;
const tip = 100;
const friends = 4;

let totalAmount = foodBill + tip;
let perPerson = totalAmount / friends;

console.log("Each person pays: ₹" + perPerson);
// Output: Each person pays: ₹475
```

javascript

```javascript
let likes = 98;

likes++;
console.log("Likes:", likes);
// Output: Likes: 99

likes++;
console.log("Likes:", likes);
// Output: Likes: 100

likes++;
console.log("Likes:", likes);
// Output: Likes: 101
```

(L)

(M)

```javascript
let stock = 5;

stock--;
console.log("Stock:", stock);
// Output: Stock: 4

stock--;
console.log("Stock:", stock);
// Output: Stock: 3

stock--;
console.log("Stock:", stock);
// Output: Stock: 2
```

Comparison Operators

The comparison operators in JavaScript compare two variables or values and return either true or false (which is - boolean value), based on comparison result.

Operator	Description	Example
==	Equal	x == y
!=	Not Equal	x != y
===	Strict equality (equal value and equal type)	x === y
!==	Strict inequality (not equal value or not equal type)	x !== y
>	Greater than	x > y
<	Less than	x < y<
>=	Greater than or Equal to	x >= y
<=	Less than or Equal to	x <= y

```
// 1. == (Equal to)
let a = 10;
let b = "10";

console.log(a == b); // true


// 2. != (Not Equal to)
let c = 10;
let d = 20;

console.log(c != d); // true


// 3. === (Strict Equal to)
let e = 10;
let f = "10";

console.log(e === f); // false
```

```
// 4. !== (Strict Not Equal to)
let g = 10;
let h = "10";

console.log(g !== h); // true


// 5. > (Greater Than)
let age = 20;
let minimumAge = 18;

console.log(age > minimumAge); // true


// 6. < (Less Than)
let marks = 40;
let passingMarks = 50;

console.log(marks < passingMarks); // true
```

```
// 7. >= (Greater Than or Equal to)
let score = 80;
let requiredScore = 80;

console.log(score >= requiredScore); // true


// 8. <= (Less Than or Equal to)
let price = 500;
let budget = 1000;

console.log(price <= budget); // true
```

Important Difference: == vs ===

```javascript
let x = 10;
let y = "10";

console.log(x == y);  // true
console.log(x === y); // false
```

Logical Operators

The **logical** operators in JavaScript are generally used with Boolean operands and return a boolean value. There are mainly three types on logical operators in JavaScript

Operator	Description	Example
&&	Logical AND	(x && y) is false.
||	Logical OR	(x || y) is true.
!	Logical NOT	!(x) is false.

These operators are used to control the flow the program.

```javascript
// 1. Logical AND (&&)
let a =  true;
let b = false;

console.log(a && b); // false


// 2. Logical OR (||)
let x = true;
let y = false;

console.log(x || y); // true


// 3. Logical NOT (!)
let p = true;

console.log(!p); // false
```

Conditional Operator (Ternary Operator)

The Ternary Operator in JavaScript is a conditional operator that evaluates a condition and returns one of two values based on whether the condition is true or false. It simplifies decision-making in code, making it more concise and readable.

Syntax :

Condition ? trueExpression : falseExpression

```
// JavaScript to illustrate Conditional operator
let PMarks = 50;
let res = (PMarks > 39) ? "Pass" : "Fail";

console.log(res);
```

javascript

```
const age = 20;
const result = age >= 18 ? "Adult" : "Minor";
console.log(result);
```

javascript

```
const product = { name: "Laptop", inStock: false };
const status = product.inStock ? "In Stock" : "Out of Stock";
console.log(`${product.name}: ${status}`);
```

javascript

```
const marks = 72;
const grade = marks >= 90 ? "A" : marks >= 60 ? "B" : "C";
console.log(grade);
```

Chapter 5: JavaScript Control Flow Statement or Conditional Statement

JavaScript conditional statements are used to make decisions in a program based on given conditions. They control the flow of execution by running different code blocks depending on whether a condition is true or false.

- Conditions are evaluated using comparison and logical operators.

Types of Conditional Statement

1. If Statement

The if statement checks a condition written inside parentheses. If the condition evaluates to true, the code inside { } is executed; otherwise, it is skipped.

```
if (condition) {
  // code runs if condition is true
}
```

```
let x = 20;

if (x % 2 === 0) {
    console.log("Even");
}

if (x % 2 !== 0) {
    console.log("Odd");
};
```

2. If-else Statement

The if executes one block of code if a condition is true and another block if it is false. It ensures that exactly one of the two code blocks runs.

- Used when there are two possible outcomes.
- The else block runs when the if condition is not satisfied.

```
if (condition)
{
    // Executes this block if
    // condition is true
}
else
{
    // Executes this block if
    // condition is false
}
```

```
let age = 25;

if (age >= 18) {
    console.log("Adult")
} else {
    console.log("Not an Adult")
};
```

3. else if Statement

The else if statement is used to test multiple conditions in sequence. It executes the first block whose condition evaluates to true.

```
const x = 0;

if (x > 0) {
    console.log("Positive.");
} else if (x < 0) {
    console.log("Negative.");
} else {
    console.log("Zero.");
};
```

```
let score = 85;
let grade;

if (score >= 90) {
  grade = "A";
} else if (score >= 80) {
  grade = "B"; // This block will run because 85 >= 80
} else if (score >= 70) {
  grade = "C";
} else if (score >= 60) {
  grade = "D";
} else {
  grade = "F"; // Runs only if score is below 60
}

console.log(`Your grade is: ${grade}`);
// Output: Your grade is: B
```

4. Nested if…else

A nested if...else statement is an if...else block written inside another if or else. It is used to evaluate multiple related conditions in a hierarchical manner.

- Useful for handling complex decision-making logic.
- Deep nesting should be avoided to maintain code readability.

```
let weather = "sunny";
let temp = 25;

if (weather === "sunny") {
    if (temp > 30) {
        console.log("It's a hot day!");
    } else if (temp > 20) {
        console.log("It's a warm day.");
    } else {
        console.log("It's a bit cool today.");
    }
} else if (weather === "rainy") {
    console.log("Don't forget your umbrella!");
} else {
    console.log("Check the weather forecast!");
};
```

5. Switch Case Statement

The switch statement evaluates an expression and executes the matching case block based on its value. It provides a clean and readable way to handle multiple conditions for a single variable.

- Used when one variable needs to be compared against multiple fixed values.
- Improves readability compared to long if...else if chains.

```
const marks = 85;

let Branch;

switch (true) {
    case marks >= 90:
        Branch = "Computer science engineering";
        break;
    case marks >= 80:
        Branch = "Mechanical engineering";
        break;
    case marks >= 70:
        Branch = "Chemical engineering";
        break;
    case marks >= 60:
        Branch = "Electronics and communication";
        break;
    case marks >= 50:
        Branch = "Civil engineering";
        break;
    default:
        Branch = "Bio technology";
        break;
}

console.log(`Student Branch name is : ${Branch}`);
```

Chapter 6: Javascript Array and its Methods

In JavaScript, an array is an ordered list of values. Each value, known as an element, is assigned a numeric position in the array called its index.

- The indexing starts at 0, so the first element is at position 0, the second at position 1, and so on.
- Arrays can hold any type of data-such as numbers, strings, objects, or even other arrays-making them a flexible and essential part of JavaScript programming.

Basic Operations on JavaScript Arrays

1. Accessing Elements of an Array - Any element in the array can be accessed using the index number. The index in the arrays starts with 0.

```
// Creating an Array and Initializing with Values
let a = ["HTML", "CSS", "JS"];

// Accessing Array Elements
console.log(a[0]);
console.log(a[1]);
```

2. Accessing the Last Element of an Array

```
// Creating an Array and Initializing with Values
let a = ["HTML", "CSS", "JS"];

// Accessing Last Array Elements
let lst = a[a.length - 1];

console.log("Last Item: ", lst);
```

3. Modifying the Array Elements

```
// Creating an Array and Initializing with Values
let a = ["HTML", "CSS", "JS"];
console.log(a);

a[1]= "Bootstrap";
console.log(a);
```

4. Adding Elements to the Array

```
// Creating an Array and Initializing with Values
let a = ["HTML", "CSS", "JS"];

// Add Element to the end of Array
a.push("Node.js");

// Add Element to the beginning
a.unshift("Web Development");

console.log(a);
```

5. Removing Elements from an Array

```
// Creating an Array and Initializing with Values
let arr = ["HTML", "CSS", "JS"];
console.log("Original Array: " + arr);
//----------------------------------------------------
// Removes and returns the last element
let lst = arr.pop();
console.log("After Removing the last: " + arr);

// Removes and returns the first element
let fst = arr.shift();
console.log("After Removing the First: " + arr);
//----------------------------------------------------
// Creating an array for splice example
let arr2 = ["HTML", "CSS", "JS"];

// Removes 2 elements starting from index 1
arr2.splice(1, 2);

console.log("After Removing 2 elements starting from index 1: " + arr2);
```

6.Array Length

```
// Creating an Array and Initializing with Values
let arr = ["HTML", "CSS", "JS"];

let length = arr.length;

console.log("Array Length: " + length);
```

7. Iterating Through Array Elements

We can iterate array and **access array elements** using for loop and **forEach loop**.

```
// Creating an Array and Initializing with Values
let arr = ["HTML", "CSS", "JS"];

// Iterating through for loop
for (let i = 0; i < arr.length; i++) {
    console.log(arr[i])
}
```

Chapter 7: Javascript Looping Statement

1. JavaScript for Loop

The for loop is used when the number of iterations is known.

```
for (let i = 1; i <= 3; i++) {
    console.log(i);
}
```

- Initializes a counter, checks condition, and updates it in each iteration.
- Executes the block until the condition becomes false.

2. JavaScript while Loop

The while loop runs as long as the condition is true.

```
let i = 1;
while (i <= 3) {
    console.log(i);
    i++;
}
```

- Checks the condition before each iteration.
- Executes the block only if the condition is true.

2. JavaScript do…while Loop

The do...while loop executes the block at least once, even if the condition is false.

```
let i = 1;
do {
    console.log(i);
    i++;
} while (i <= 3);
```

- Executes the block first, then checks the condition.
- Ensures at least one execution.